Data Structures

Lecture 6

Queues

Abstract Data Type

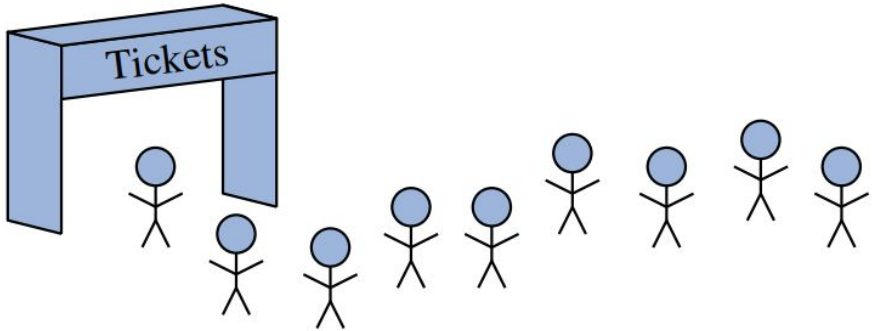
References:
Data Structures and Algorithms in Java
Michael T. Goodrich and Roberto Tamassia.

University of Kufa
Faculty of Computer Science and Mathematics

Dr. Munqath Al-Attar
Information Technology Research and Development Center ITRDC
Email:
munqith.alattar@uokufa.edu.iq
Website:
http://staff.uokufa.edu.iq/profile.html?munqith.alattar

Topics

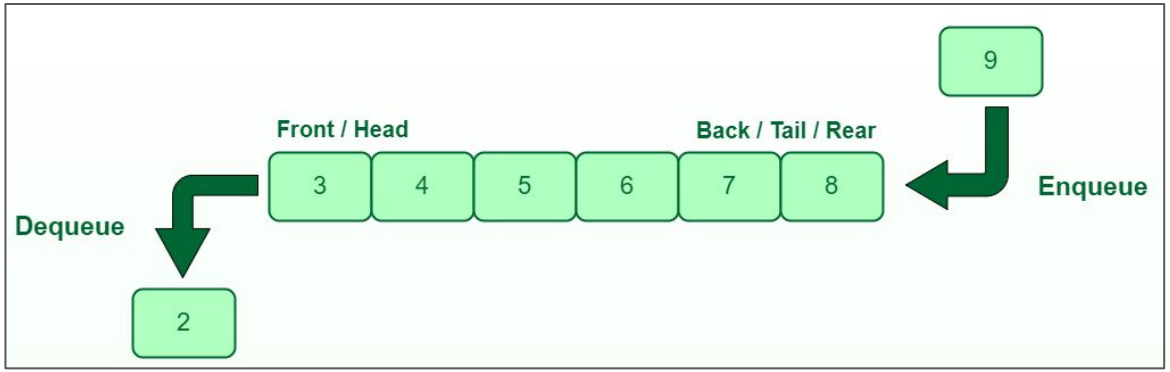
- Introduction to Queues
- Queues Operations
- Application of Queues
- Circular Queue Data Structure



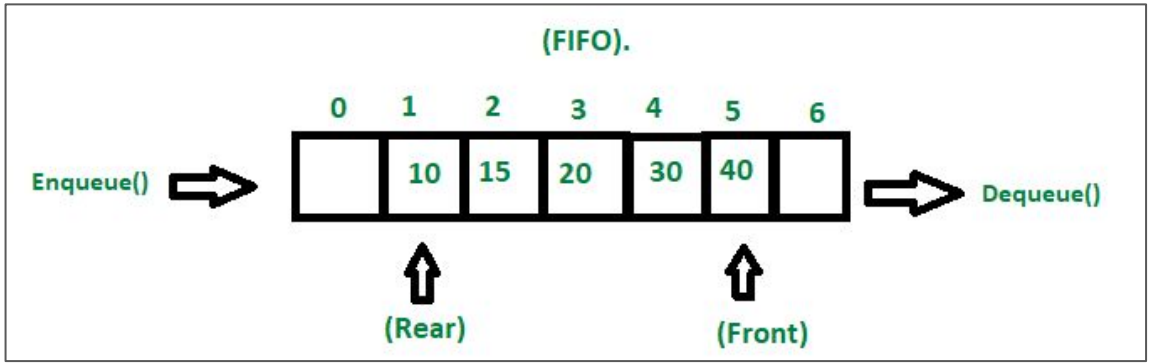
Data Structures
Introduction to Queues

- A queue is a linear data structure that is **first-in-first-out (FIFO) data structure.**
- A queue is like a line waiting to buy tickets, (first person in line is the first person to buy)
- A queue is open at both ends, where the addition of entities is at one end and the removal of entities is from the other end of the queue.

- It can handle multiple data.
- Both ends can be accessed.
- A queue is fast and flexible.



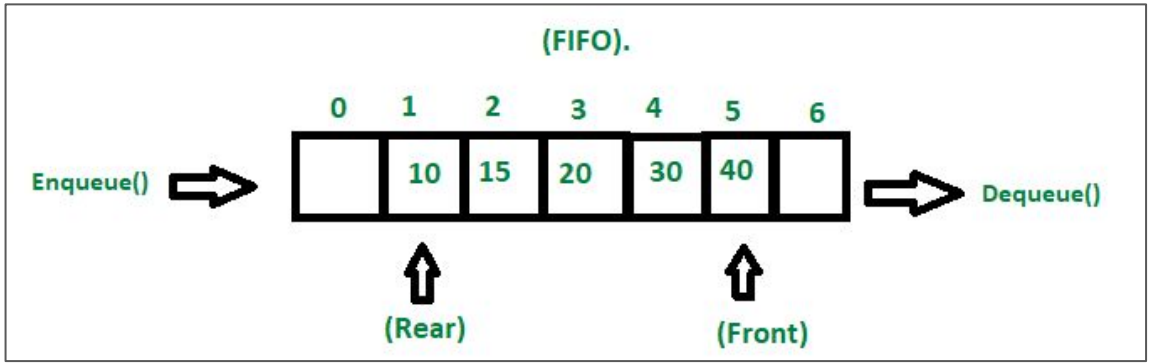
- **Back (tail, rear):** the end of the queue at which elements are added.
- **Front (head):** the end at which elements are removed.



Queue Representation:

Queues can be represented in an array, variables:

- **Queue**: the array storing queue elements.
- **Front (head):** the index where the first element is stored.
- **Back (tail, rear):**: the index where the last element is stored.



Data Structures
Queues Operations

- Initially, FRONT and REAR = -1

enqueue(e):	Adds element e to the back of queue.
dequeue( ):	Removes and returns the first element from the queue (or null if the queue is empty).
first( ):	Returns the first element of the queue, without removing it (or null if the queue is empty).
size( ):	Returns the number of elements in the queue.
isEmpty( ):	Returns a boolean indicating whether the queue is empty.
Peek (front):	Returns the value of the next element to be dequeued without removing it.

- All the operations take O(1) time.

Data Structures
Queues Operations

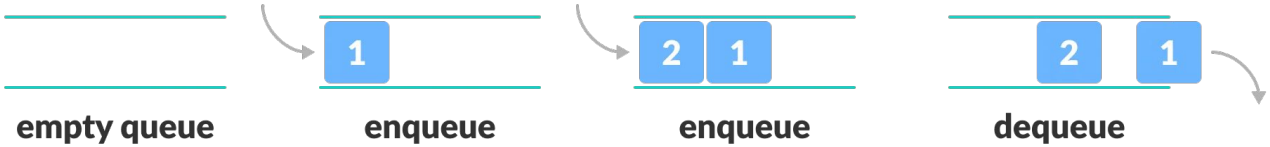
Enqueue Operation

- Check if the **queue** is full
- For the first element, set **FRONT** = 0
- Increase the **REAR** by 1
- Add the new element in the position pointed by **REAR**

Dequeue Operation

- Check if the **queue** is empty
- Return the value pointed by **FRONT**
- Increase the **FRONT** by 1
- For the last element, reset **FRONT** and **REAR** to -1

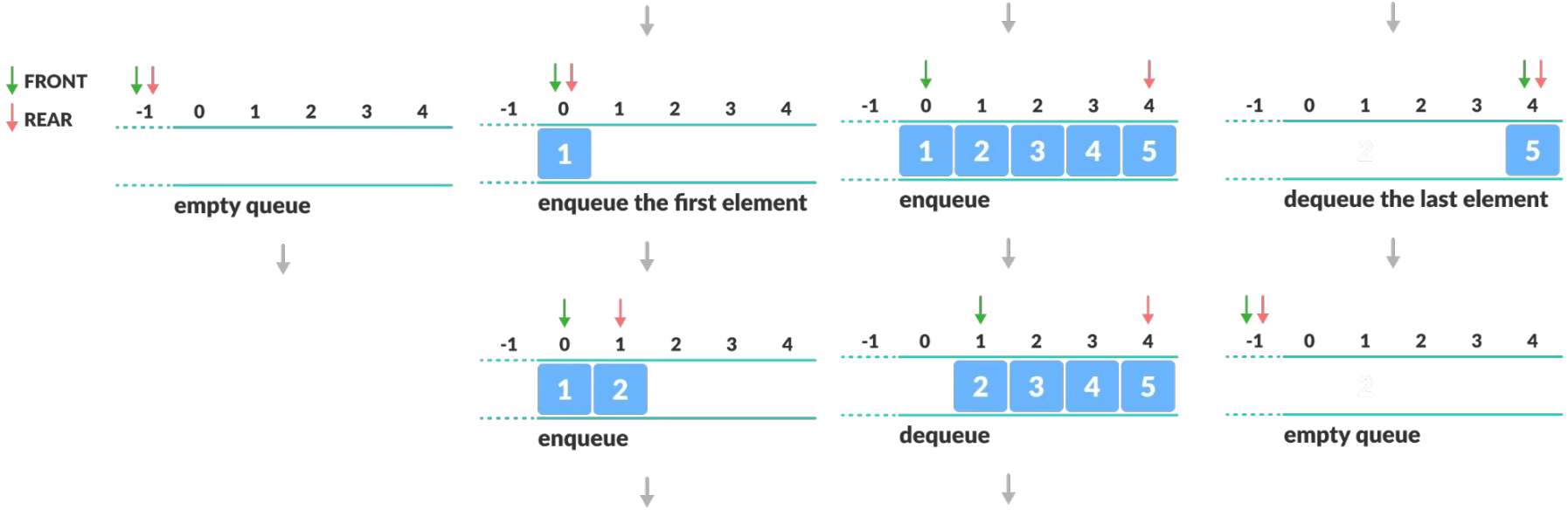
Data Structures
Queues Operations



Example 6.4: *The following table shows a series of queue operations and their effects on an initially empty queue Q of integers.*

Method	Return Value	first ← Q ← last
enqueue(5)	–	(5)
enqueue(3)	–	(5, 3)
size( )	2	(5, 3)
dequeue( )	5	(3)
isEmpty( )	false	(3)
dequeue( )	3	( )
isEmpty( )	true	( )
dequeue( )	null	( )
enqueue(7)	–	(7)
enqueue(9)	–	(7, 9)
first( )	7	(7, 9)
enqueue(4)	–	(7, 9, 4)

Data Structures
Queues Operations

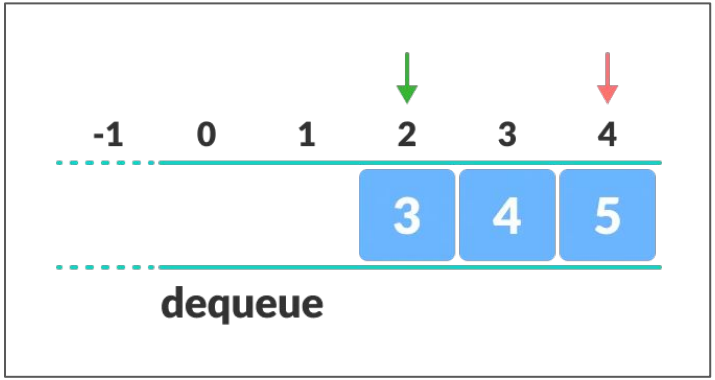


Applications of Queue

- CPU scheduling
- Disk Scheduling
- Waiting lists for a single shared resource (Printing documents).
- Used in Call Center systems, to hold people and calling them in order.
- Breadth-First Search implementation.

Limitations of Queue

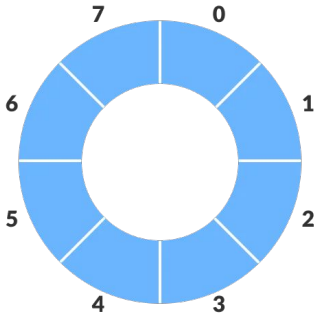
- Searching an element takes O(N) time.
- after some enqueuing and dequeuing, there will be non-usable empty space.
  - Indexes 0 and 1 can only be used after deletion of all elements.
  - This reduces the actual size of the queue.



Circular queue solves the major limitation of the queue of having non-usable empty space.

Circular queue is the extended version of the queue where the **last element is connected to the first element.**

In Circular Queue, when incrementing the pointer and it reach the end of the queue, it start from the beginning of the queue.



- The circular increment is done by ***modulo division*** with the queue size:

```
if REAR + 1 == QSize (overflow!), REAR = (REAR + 1)%QSize = 0 (start of queue)
```